European Journal of Operational Research 330 (2026) 995–1008

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/eor

Innovative Applications of O.R.

The Vehicle Routing Problem with Pickups, Deliveries, and Scheduled Linehauls

Casper H. Bazelmans *,[1], Siem L. ter Braake[1], Albert H. Schrotenboer, Rolf N. van Lieshout, Tom Van Woensel

Department of Industrial Engineering & Innovation Sciences, Eindhoven University of Technology, 5600 MB Eindhoven, The Netherlands

ARTICLE INFO

Keywords:
Routing
Pickup and delivery problem
Column generation
Large neighborhood search
Real-life case study

ABSTRACT

We study the joint collection and distribution of less-than-truckload freight in a multi-depot setting. Linehauls between the depots allow for an increased freight consolidation, saving cost and greenhouse gas emissions. However, to realize these benefits, the linehauls must be coordinated with daily pickup-and-delivery routes from the depots to collect and distribute products. The routes are subject to loading constraints, which restrict pickups and deliveries based on the vehicles' distribution and collection loads. In collaboration with a major European logistics provider, we define this problem as the Vehicle Routing Problem with Pickups, Deliveries, and Scheduled Linehauls. We propose a set-covering formulation and solve this via a novel restricted master heuristic where the associated labeling algorithm effectively incorporates the time synchronization between the departure times of generated routes and associated linehauls. Moreover, we develop an adaptive large neighborhood search (ALNS) algorithm that finds high-quality solutions quickly. The ALNS also populates the route sets and provides initial solutions within the restricted master heuristic. Computational results show that for instances with up to 45 customers, the restricted master heuristic enforced with the ALNS finds solutions with only a 0.04% gap relative to the linear relaxation at the root node. A case study at our industry partner based on 24 days of operations reveals that our methods can save up to 17.4% of costs. Additionally, we explore the impact of implementing service area restrictions per depot, a common industry practice that simplifies operations by transforming the problem into single-depot variants. Our findings show that this results in an average cost increase of 4.9%.

1. Introduction

The joint collection and distribution of less-than-truckload freight is a fundamental operation for major logistics service providers (LSPs). In addition to first-mile collection and last-mile distribution, LSPs that operate internationally also employ linehauls to connect depots in different regions. These linehauls enable locally collected freight to be sent to international destinations efficiently, and vice versa, so that internationally collected freight can be distributed efficiently within the local area. In some sense, these local depots act as cross-docks within their local operations. To save costs, it is crucial to coordinate the linehaul operations with the daily vehicle routes for collection and distribution.

This paper is the result of a collaboration with a major European logistics service provider. Their main operations are as follows. Our industry partner receives thousands of transportation requests from their local area daily, and approximately half of these requests originate from the international network. Such requests arrive using an international linehaul to one of the depots in the local area and should be delivered to customers on the same day. The other half of the requests need to be picked up at customer locations within the local area. They are designated to one of the local area's depots, from which they are transported internationally. Linehauls between local depots can further optimize last-mile collection and distribution by consolidating freight between them to increase truck fill rates. To streamline operations, the vehicle routes exhibit mixing levels. Requests can only be picked up if at least a fraction of the truck's capacity is available, and vice versa; requests must be delivered before a certain quantity of pickups are made. Fig. 1 presents a stylized illustration of these operations.

The operations described above resemble a combination of two classic vehicle routing problems: the pickup-and-delivery problem with

* Corresponding author.
E-mail addresses: c.h.bazelmans@tue.nl (C.H. Bazelmans), s.l.t.braake@student.tue.nl (S.L.t. Braake), a.h.schrotenboer@tue.nl (A.H. Schrotenboer), r.n.v.lieshout@tue.nl (R.N. van Lieshout), t.v.woensel@tue.nl (T. Van Woensel).
[1] These authors contributed equally to this work.

https://doi.org/10.1016/j.ejor.2025.07.070
Received 30 August 2024; Accepted 30 July 2025
Available online 23 August 2025
0377-2217/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

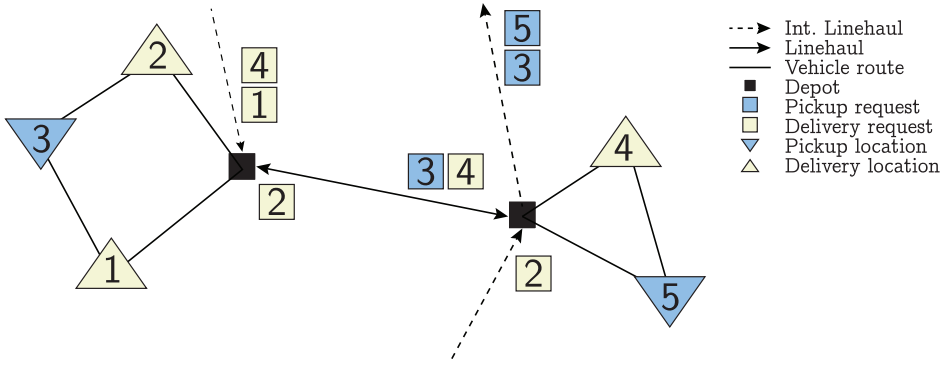


Fig. 1. Illustration of the operations at our industry partner.
Note. Delivery requests 1, 2, and 4 arrive in the network via international linehauls. Request 1 is transported directly by vehicle to the corresponding customer node, while requests 2 and 4 are sent via a linehaul. Pickup requests 3 and 5 are collected in a vehicle route. Pickup requests have a destination depot from which an international linehaul departs. Request 5 is picked up in a direct vehicle route of its destination depot, while request 3 takes a linehaul.

transfers (Cortés, Matamala, & Contardo, 2010) and the vehicle routing problem with simultaneous pickups and deliveries (Min, 1989). Indeed, the operations at our industry partner consider transfers (i.e., linehauls) and simultaneous pickups and deliveries—a combination of characteristics that has not yet been discussed in the existing literature, despite its practical relevance. To our knowledge, no tailored exact and heuristic methods have been developed for such problems, especially not considering various operational constraints. For example, the mixing levels fundamentally alter the structure of feasible pickup-and-delivery routes. This potentially impacts the efficacy of transfers in pickup-and-delivery problems, and thus, further attention is required to evaluate the need for transfers and linehauls. Furthermore, our collaboration with the industry partner revealed the need to solve large-scale instances of up to 2500 collection and distribution requests in a single day, which is unprecedented in the current literature when considering rich vehicle routing problem variants.

In this paper, we introduce the Vehicle Routing Problem with Pickups, Deliveries, and Scheduled Linehauls (VRPPDL). The VRPPDL aims to find a cost-minimizing set of vehicle routes and an assignment of transportation requests to linehauls while considering various operational constraints, including vehicle and linehaul capacities, mixing levels, time synchronization, customer time windows, and maximum vehicle driving and working times. We first model the VRPPDL as a compact mixed-integer linear program. Then, we use a Dantzig–Wolfe decomposition among the vehicle types and depots to arrive at a set-covering formulation, where each column represents a vehicle route and the assignment of its associated customers to linehauls. The subproblem is an elementary shortest path problem with resource and time synchronization constraints, for which we develop a tailored labeling algorithm to solve it optimally. Specifically, the labeling algorithm incorporates a departure time interval for each partial path, allowing for the optimal assignment of customers to linehauls while dynamically expanding the partial path. The restricted master heuristic employs the labeling algorithm to solve the linear relaxation of the root node to optimality, and afterward finds an integer solution via branch-and-bound with the generated columns using a commercial solver. This approach is often shown to reach high-quality solutions, especially when the root-node optimality gaps are small (see, e.g., Sonntag, Schrotenboer, & Kiesmüller, 2023).

In addition to the restricted master heuristic and the associated labeling algorithm, we also develop a tailored adaptive large neighborhood search (ALNS) algorithm for solving the VRPPDL. The ALNS serves three main purposes. First, we use it standalone to solve practically-sized large-scale instances. Second, we use the solutions obtained from the ALNS as initial solutions for the restricted master heuristic. This helps stabilize the dual values during the column generation process within our labeling algorithm, reducing the number of unnecessary generated routes. Third, we employ the ALNS to populate the route set during the column generation by heuristically solving the elementary shortest path problem with resource and time synchronization constraints. Compared to the existing literature (see, e.g., Subramanian, Uchoa, & Ochi, 2013), we extend the framework for integrating metaheuristic algorithms in column generation approaches to include a metaheuristically accelerated exact column generation approach.

On a structured set of benchmark instances, we show the performance of our suite of algorithms. We consider five algorithms: four variants of our restricted master heuristic that incorporate the ALNS algorithm and the exact labeling algorithm in different ways, and one that employs the ALNS algorithm alone. Our results reveal that employing ALNS to both initialize the route set and generate reduced cost columns reduces the computational time of our column generation algorithm by 98%. Furthermore, when we omit the labeling algorithm and generate routes exclusively using the ALNS, the resulting lower bounds are within 0.04% of the root node lower bounds—demonstrating that pricing via ALNS is efficient as it finds most of the routes appearing in the optimal root-node relaxation. Additionally, the standalone ALNS algorithm yields competitive solutions for medium- to large-scale benchmark instances on a special case of the VRPPDL from the literature.

Finally, we present a case study at our industry partner to show the impact and effectiveness of our approach to solving the VRPPDL. First, we compare our algorithms for solving the VRPPDL with our industry partner's current operations. Over 24 days of operation, we demonstrate a 17.4% decrease in costs. Additionally, we utilize our algorithms to analyze a common practical approach employed by logistics service providers operating from multiple depots. Specifically, they assign a specific geographical area to each depot, thereby treating the multi-depot problem as multiple single-depot problems. We demonstrate that this practice results in a 4.9% cost increase at our industry partner. Therefore, we advocate integrating decision-making among multiple depots in practice. All data, including the large-scale real-life VRPPDL instances, and the source code can be found in our Github repository.[2] We encourage other researchers and practitioners to try their hand at these instances, as we believe this will further progress the understanding of solving large-scale real-life vehicle routing problems and associated algorithm design.

The remainder of this paper is as follows. Section 2 examines literature related to the VRPPDL, and Section 3 introduces a mixed-integer programming formulation for the problem. Section 4 formulates a set-covering formulation and describes how routes that incorporate the time synchronization between the route and linehauls are generated. Section 5 introduces an ALNS metaheuristic for the VRPPDL,

[2] github.com/siemterbraake/vrppdl-algorithms-instances

and Section 6 presents computational results for all our algorithms. Section 7 presents a case study at our industry partner and provides managerial insights, followed by conclusions in Section 8.

2. Literature review

In this section, we first review the literature on the pickup and delivery problem with transfers (PDP-T), as the VRPPDL belongs to this broad class of vehicle routing problems. Second, we investigate the vehicle routing problem with simultaneous pickup and delivery (VRPSPD), a single-depot variant of the VRPPDL that does not involve linehauls or mixing levels.

2.1. Pickup and delivery problems with transfers

Pickup and delivery problems (PDPs) generalize the vehicle routing problem by considering forward and reverse logistics within single-vehicle routes. Following the classification by Berbeglia et al. (2007), the VRPPDL is a one-to-many-to-one PDP. In such problems, delivery shipments are moved from a central depot to customer nodes, while pickup shipments available to customers are moved back to the central depot. A transfer in the context of PDPs is defined as moving freight between vehicles. It aims to improve consolidation, truck fill rates, and cost savings. Previous work on transfers in the context of PDPs is limited to so-called one-to-one PDPs.

The one-to-one pickup and delivery problem with time windows and transfers (PDPTW-T) is first described in a mixed-integer programming (MIP) formulation by Cortés et al. (2010). Recently, Lyu and Yu (2023) proposed a new, improved formulation. This formulation solves instances of up to 25 nodes with a commercial MIP solver. Generally speaking, transfers complicate the problem structure as routes are no longer independent. As a result, only small-scale instances can be solved with compact MIP formulations (Drexl, 2012).

In the PDPTW-T, transfers are integrated into the routing problem, and the decision-maker determines the transfer location and time. However, in real-world transportation networks, transfers often follow a predefined schedule, only allowing them at predefined locations and times. Ghilas, Demir, and Van Woensel (2016b) introduce the pickup and delivery problem with scheduled lines (PDPTW-SL), where transfers follow a fixed schedule. Scheduled lines are equivalent to the concept of linehauls in this study. Ghilas, Cordeau, Demir, and Van Woensel (2018) introduce an exact branch-and-price algorithm that solves instances of the PDPTW-SL with up to 50 nodes. An ALNS metaheuristic by Ghilas, Demir, and Van Woensel (2016a) solves medium-sized PDPTW-SL instances up to 100 nodes.

Similar to the PDPTW-T, the VRPPDL includes time windows and transfers. However, it is a one-to-many-to-one PDP. This decouples a shipment's pickup and delivery nodes, assuming they are connected through a central depot. Furthermore, the VRPPDL includes mixing levels to ensure that mixing pickup and delivery shipments is feasible. Moreover, research on the PDPTW-T has been limited to small to medium-sized instances. For the VRPPDL, we develop a metaheuristic that solves realistic instances of up to 2500 nodes.

2.2. Vehicle routing problems with simultaneous pickup and delivery

The most frequently studied one-to-many-to-one PDP is the vehicle routing problem with simultaneous pickup and delivery (VRPSPD), as introduced by Min (1989). Angelelli and Mansini (2002) extend this problem to include time windows and apply a branch-and-price algorithm to solve small problem instances. Nagy and Salhi (2005) first consider the multi-depot case of this problem, where multiple heuristics are presented.

Dell'Amico, Righini, and Salani (2006) introduce an exact branch-and-price algorithm for the VRPSPD, which solves instances of up to 40 nodes. Subramanian, Uchoa, Pessoa, and Ochi (2013) introduce a branch-cut-and-price algorithm for the VRPSPD that solves instances of up to 200 nodes. Metaheuristics are applied to solve medium- to large-scale problem instances efficiently. Koç, Laporte, and Tükenmez (2020) provide an extensive overview of metaheuristics applied to the VRPSPD. They conclude that local search-based metaheuristics such as Avci and Topaloglu (2016) and Goksal, Karaoglan, and Altiparmak (2013) yield better solutions than other approaches. A recent study by Cavaliere, Accorsi, Laganà, Musmanno, and Vigo (2024) introduces an iterated local search procedure for the VRPSPD that scales linearly in the instance size, which is applied to instances of up to 30,000 nodes.

The VRPPDL generalized the VRPSPD by incorporating mixing levels, time windows, and linehauls. With these problem additions, we aim to advance research on realistic routing problems experienced by logistics companies in the industry.

3. Problem formulation

This section presents a mixed-integer linear program for the VRPPDL and proposes a decomposed set-covering formulation.

3.1. Mixed-integer programming formulation

Let $G = (N, A)$ be a directed graph with a set of nodes N and a set of arcs $A = \{(i,j) | i,j \in N, i \neq j\}$. The set of nodes N consists of depot nodes N_0 and customer nodes N_c, where $N = N_0 \cup N_c$. The set of customer nodes N_c contains pickup nodes N_p and delivery nodes N_d so that $N_c = N_p \cup N_d$. Subset $E \subset A$ contains all arcs between depot nodes $E = \{(i,j) | i,j \in N_0, i \neq j\}$, which are solely used by linehauls.

Let T represent the length of the time horizon. On this time horizon, we fulfill a set of requests $R = R_p \cup R_d$, divided into collection and distribution requests, respectively. For collection requests, we pick up the request at customer $p(r) \in N_p$ and deliver it to the delivery depot $d(r) \in N_0$ before the deadline o^r. Similarly, for distribution requests, we pick up the request at the pickup depot $p(r) \in N_0$, after its release time b^r, and deliver it to the delivery customer $d(r) \in N_d$. So, each request has an associated depot $\mu^r \in N_0$. Moreover, each customer location $i \in N_c$ has a time window $[e_i, \ell_i]$ in which its service, of length p_i, must be completed.

Customers are served by a heterogeneous fleet of vehicles V. Each vehicle $v \in V$ starts and ends at depot $\mu^v \in N_0$, and has vehicle type $\kappa^v \in K$ where K is the set of vehicle types. The number of available vehicles of type $k \in K$ at depot $i \in N_0$ is given by M_i^k. The route duration of each vehicle is limited by the maximum driving time h^v and working time g^v, which also includes service and waiting time. Each vehicle v has a maximum capacity Q^v, and two thresholds: α^v and β^v, that manage the balance between collections and distributions. Specifically, α^v is the maximum distribution load a vehicle can carry while still being allowed to pick up additional freight; as long as the total distribution load exceeds α^v, the vehicle cannot perform collection operations. Conversely, β^v represents the maximum collection load a vehicle can have before it is restricted from making further deliveries. We collectively call α^v and β^v the mixing levels. The cost and driving time of traveling arc $(i,j) \in A$ are given by δ_{ij}^v and τ_{ij}^v for each vehicle v, respectively.

We denote S as the set of scheduled linehauls between depots. Linehaul, $s \in S$, leaves its origin depot at departure time d_s and arrives at its destination depot at arrival time a_s. Transporting a single unit of quantity with a linehaul costs f_s, and each linehaul has a maximum capacity of Q_s. A collection request $r \in R_p$ can only be assigned to linehauls that depart from μ^r after release time b^r. Conversely, a distribution request $r \in R_d$ can only be assigned to linehauls that arrive at μ^r before deadline o^r. We denote S_r as the set of feasible linehauls requests that can be assigned to $r \in R$. For modeling reasons, this set also includes a dummy linehaul, specifically for request $r \in R$, that arrives and departs at μ^r at time o^r for collection requests and time b^r for distribution requests. The request is assigned to this dummy linehaul

if the request does not take a linehaul. Finally, let $S_r^-(i) \subseteq S_r$ and $S_r^+(i) \subseteq S_r$ be the linehauls arriving at and departing from depot $i \in N_0$, respectively.

The VRPPDL aims to find a cost-minimizing set of routes and assignment of transportation requests to linehauls while considering vehicle and linehaul capacities, mixing levels, time synchronization, customer time windows, and the maximum vehicle driving and working times.

We introduce the following decision variables to model the VRPPDL. Binary decision variable x_{ij}^r represents individual request flows and equals 1 if request $r \in R$ is transported over arc $(i,j) \in A$, 0 otherwise. Furthermore, binary decision variable y_{ij}^v captures vehicle flow and takes value 1 if vehicle $v \in V$ drives arc $(i,j) \in A$, 0 otherwise. The departure time of vehicle $v \in V$ at customer node $i \in N_c$ is captured by the continuous decision variable t_i^v. The departure and arrival time of the vehicle $v \in V$ at depot μ^v are modeled via the continuous decision variables t_{START}^v and t_{END}^v. Lastly, the binary decision variable z_s^r takes the value 1 if request $r \in R$ is assigned to linehaul $s \in S_r$ and equals 0 otherwise. Using these variables, the VRPPDL is formulated as a mixed-integer linear program:

$$\min \sum_{r\in R}\sum_{s\in S} f_s z_s^r + \sum_{i\in N}\sum_{j\in N}\sum_{v\in V} \delta_{ij}^v y_{ij}^v, \tag{1}$$

$$\text{s.t.} \sum_{j\in N} x_{p(r),j}^r = 1 \qquad \forall r \in R_p, \tag{2}$$

$$\sum_{j\in N} x_{ji}^r - \sum_{j\in N} x_{ij}^r = 0 \qquad \forall r \in R_p,\ i \in N_c \setminus \{p(r)\}, \tag{3}$$

$$\sum_{j\in N} x_{ji}^r = \sum_{s\in S_r^+(i)} z_s^r \qquad \forall r \in R_p,\ i \in N_0, \tag{4}$$

$$\sum_{j\in N} x_{ij}^r = \sum_{s\in S_r^-(i)} z_s^r \qquad \forall r \in R_d,\ i \in N_0, \tag{5}$$

$$\sum_{j\in N} x_{ji}^r - \sum_{j\in N} x_{ij}^r = 0 \qquad \forall r \in R_d,\ i \in N_c \setminus \{d(r)\}, \tag{6}$$

$$\sum_{j\in N} x_{j,d(r)}^r = 1 \qquad \forall r \in R_d, \tag{7}$$

$$\sum_{j\in N} y_{j\mu^v}^v = \sum_{j\in N} y_{\mu^v j}^v = 1 \qquad \forall v \in V, \tag{8}$$

$$\sum_{j\in N} y_{ji}^v - \sum_{j\in N} y_{ij}^v = 0 \qquad \forall v \in V,\ i \in N, \tag{9}$$

$$\sum_{j\in N}\sum_{v\in V} y_{ji}^v = \sum_{j\in N}\sum_{v\in V} y_{ij}^v = 1 \qquad \forall i \in N_c, \tag{10}$$

$$\sum_{j\in N} y_{ji}^v = 0 \qquad \forall v \in V,\ i \in N_0 \setminus \{\mu^v\}, \tag{11}$$

$$\sum_{r\in R} x_{ij}^r q^r \le \sum_{v\in V} y_{ij}^v Q^v \qquad \forall (i,j) \in A \setminus E, \tag{12}$$

$$\sum_{r\in R_d}\sum_{j\in N} x_{ji}^r q^r \le \alpha^v + M\left(1 - \sum_{j\in N} y_{ji}^v\right) \qquad \forall v \in V,\ i \in N_p, \tag{13}$$

$$\sum_{r\in R_p}\sum_{j\in N} x_{ij}^r q^r \le \beta^v + M\left(1 - \sum_{j\in N} y_{ij}^v\right) \qquad \forall v \in V,\ i \in N_d, \tag{14}$$

$$\sum_{i\in N}\sum_{j\in N} y_{ij}^v \tau_{ij}^v \le h^v \qquad \forall v \in V, \tag{15}$$

$$t_{\text{END}}^v - t_{\text{START}}^v \le g^v \qquad \forall v \in V, \tag{16}$$

$$\sum_{s\in S_r} z_s^r = 1 \qquad \forall r \in R, \tag{17}$$

$$\sum_{r\in R} z_s^r q^r \le Q_s \qquad \forall s \in S, \tag{18}$$

$$t_i^v + \tau_{ij}^v + p_j \le t_j^v + M(1 - y_{ij}^v) \qquad \forall v \in V,\ i,j \in N_c, \tag{19}$$

$$t_{\text{START}}^v + \tau_{\mu^v i}^v + p_i \le t_i^v + M(1 - y_{\mu^v i}^v) \qquad \forall v \in V,\ i \in N_c, \tag{20}$$

$$t_i^v + \tau_{i\mu^v}^v \le t_{\text{END}}^v + M(1 - y_{i\mu^v}^v) \qquad \forall v \in V,\ i \in N_c, \tag{21}$$

$$e_i + p_i \le t_i^v \qquad \forall v \in V,\ i \in N_c, \tag{22}$$

$$t_i^v \le \ell_i \qquad \forall v \in V,\ i \in N_c, \tag{23}$$

$$t_{\text{START}}^v \ge z_s^r a_s - M\left(1 - \sum_{j\in N} y_{j,d(r)}^v\right) \qquad \forall v \in V,\ r \in R_d,\ s \in S_r, \tag{24}$$

$$t_{\text{END}}^v \le T - z_s^r(T - d_s) + M\left(1 - \sum_{j\in N} y_{p(r),j}^v\right) \qquad \forall v \in V,\ r \in R_p,\ s \in S_r, \tag{25}$$

$$x_{ij}^r \in \{0,1\} \qquad \forall r \in R,\ i,j \in N, \tag{26}$$

$$y_{ij}^v \in \{0,1\} \qquad \forall v \in V,\ i,j \in N, \tag{27}$$

$$z_s^r \in \{0,1\} \qquad \forall r \in R,\ s \in S_r, \tag{28}$$

$$0 \le t_i^v \le T \qquad \forall v \in V,\ i \in N_c, \tag{29}$$

$$0 \le t_{\text{START}}^v \le T \qquad \forall v \in V, \tag{30}$$

$$0 \le t_{\text{END}}^v \le T \qquad \forall v \in V. \tag{31}$$

The objective function (1) minimizes the cost of using linehauls and driving vehicles. Constraints (2)–(7) ensure request flow conservation for collection and distribution requests separately. Specifically, constraints (4) and (5) ensure that request flows arrive and depart from the depot of their assigned linehaul for pickup and delivery customers, respectively. Constraints (8)–(11) ensure that vehicles start and end at their associated depot, vehicle flow is conserved, each customer is only visited by a single vehicle, and that vehicles do not visit foreign depot nodes. Constraints (12)–(14) are vehicle capacity constraints for the maximum capacity, mixing quantity, and fill-up-to quantity for each vehicle. Here, M is a sufficiently large number. Constraints (15) and (16) ensure the maximum drive and work time for each vehicle, respectively. Constraint (17) ensures every request is assigned to exactly one (dummy) linehaul, and constraint (18) ensures the linehaul capacity is respected. Constraints (19)–(21) enforce vehicle time flow, and constraints (22) and (23) ensure customers are served in their time window. Constraints (24) and (25) ensure that the start and end time of a route are restricted by the (dummy) linehaul assignments of the customers in each route. Finally, constraints (26) to (31) indicate the domain of the decision variables.

3.2. Set-covering formulation

An alternative formulation for the VRPPDL is a set-covering formulation. Let Ω be the set of all feasible routes in the VRPPDL, and let $\Omega_{ik} \subset \Omega$ contain routes for vehicle type $k \in K$ associated with depot $i \in N_0$. For each route $u \in \Omega$, let c^u be the total costs, consisting of the routing costs and the linehaul assignment costs. Also, let θ_j^u be a binary parameter indicating if route $u \in \Omega$ visits customer $j \in N_c$. Moreover, let γ_{js}^u be a binary parameter indicating if customer $j \in N_c$ visited by route $u \in \Omega$ is assigned to linehaul $s \in S$. Finally, let λ^u be a binary decision variable equal to 1 if route $u \in \Omega$ is selected and equals 0 otherwise. The VRPPDL is then formulated as follows:

$$\min \sum_{i\in N_0}\sum_{k\in K}\sum_{u\in\Omega_{ik}} c^u \lambda^u, \tag{32}$$

$$\text{s.t.} \sum_{i\in N_0}\sum_{k\in K}\sum_{u\in\Omega_{ik}} \theta_j^u \lambda^u \ge 1 \qquad \forall j \in N_c, \tag{33}$$

$$\sum_{u\in\Omega_{ik}} \lambda^u \le M_i^k \qquad \forall i \in N_0,\ \forall k \in K, \tag{34}$$

$$\sum_{i\in N_0}\sum_{k\in K}\sum_{u\in\Omega_{ik}}\sum_{j\in N_c} q_j \gamma_{js}^u \lambda^u \le Q_s \qquad \forall s \in S, \tag{35}$$

$$\lambda^u \in \{0,1\} \qquad \forall i \in N_0,\ \forall k \in K,\ \forall u \in \Omega_{ik}. \tag{36}$$

The objective function (32) minimizes the total cost of selected routes. Constraint (33) ensures that every customer is visited in at least one route. Constraint (34) restricts the number of routes selected for

each depot and vehicle type, and constraint (35) restricts this selection based on the linehaul capacity. We refer to the model described in Eqs. (32)–(36) as the master problem (MP) and obtain the linear relaxation of this problem (LMP) by replacing constraint (36) with $\lambda^u \geq 0$.

Each set Ω_{ik} contains all feasible routes for depot $i \in N_0$ and vehicle type $k \in K$, and is impractical to enumerate, even for medium-sized instances. Hence, we typically follow an iterative scheme known as column generation. Here, the LMP is solved iteratively subject to a subset of routes $\overline{\Omega}_{ik} \subset \Omega_{ik}$. We call the LMP subject to a subset of routes the restricted linear master problem (RLMP). The RLMP solution yields dual variables π_j, η_i^k, ϕ_s associated with constraints (33)–(35), respectively. These duals are used to determine whether the solution of RLMP is optimal for LMP by solving the pricing (sub)problem for each depot $i \in N_0$ and vehicle type $k \in K$.

The pricing problem is formulated as follows. For each depot $i \in N_0$ and vehicle type $k \in K$, let χ_{nj}^u be 1 if route u travels over arc $(n, j) \in A$, 0 otherwise. Moreover, let γ_{js}^u be 1 if customer $j \in N_c$ is assigned to linehaul $s \in S$ in route u, and 0 otherwise. The reduced cost of route $u \in \Omega_{ik} \backslash \overline{\Omega}_{ik}$ is defined as

$$\hat{c}^u := -\eta_i^k + \sum_{(n,j)\in A} \chi_{nj}^u \hat{\delta}_{nj}^k + \sum_{s\in S} \sum_{j\in N_c} \gamma_{js}^u q_j (f_s - \phi_s), \tag{37}$$

where we define $\hat{\delta}_{nj}^k$ as

$$\hat{\delta}_{nj}^k = \begin{cases} \delta_{nj}^k - \pi_j & \text{if } j \in N_c \\ \delta_{nj}^k & \text{if } j \notin N_c. \end{cases} \tag{38}$$

The pricing problem $PP(i,k)$ is then given by:

$$PP(i,k) := \underset{u\in\Omega_{ik}\backslash\overline{\Omega}_{ik}}{\arg\min}\ \hat{c}^u.$$

If $\hat{c}^u$ is negative, we add it to the restricted route set in the master problem and resolve the RLMP. If no negative reduced cost routes are found for any $i \in N_0$ and $k \in K$, RLMP's objective equals LMP's objective, and LMP is solved to optimality.

4. Restricted master heuristic

In this section, we define the pricing problems $PP(i,k)$ and present a tailored labeling algorithm for solving them. Because the structure of the pricing problems is similar for each $i \in N_0, k \in K$, we assume that some $i \in N_0$ and $k \in K$ are given throughout the remainder of this section. Furthermore, when referring to the parameters of vehicles, we slightly abuse notation and use vehicle type indices instead of individual vehicles. The labeling algorithm is used to solve the root node relaxation of RMP (i.e., LMP) to optimality. Afterward, we perform branch-and-bound with the routes generated in the root node only, leading to a so-called restricted master heuristic.

4.1. The pricing problem

The pricing problem $PP(i,k)$ is a new variant of the elementary shortest path problem with resource constraints (Irnich & Desaulniers, 2005). Specifically, we must consider the time synchronization between the assigned linehauls to customers and the route's start and end times. Note that the assignment of linehauls is part of our decision, so we have to dynamically control the route's start and end time for the assignment to linehauls. We refer to our $PP(i,k)$ as the elementary shortest path problem with resource and time synchronization constraints (ESPPRTSC). The ESPPRTSC is defined on a graph $\mathcal{G} = (\mathcal{N}, \mathcal{A})$ where $\mathcal{N}$ are the nodes and $\mathcal{A}$ are the arcs between the nodes. Each arc $(n, j) \in \mathcal{A}$ has a reduced cost $\hat{\delta}_{nj}^k = \delta_{nj}^k - \pi_j$. The ESPPRTSC seeks the minimum cost path from the depot over graph $\mathcal{G}$ back to the depot. Here, the minimum cost is the reduced cost of traveling the arcs in the path and assigning each visited customer to a linehaul. The dual values ϕ_s associated with the cost of a customer-linehaul assignment cannot be incorporated on the graph directly, as the customer-linehaul assignment's feasibility relates to a path's start and end time. Besides this, the ESPPRTSC has to consider the mixing levels and the discrepancy between driving time and working time. To the authors' knowledge, this complex variant of the elementary shortest path problem has not been addressed in the literature before.

4.2. Label definition

Let label L store information about a partial path starting at depot i and ending at customer node $n(L)$. Label L is defined as a tuple with the following elements:

- The partial path $\vec{N}(L)$,
- The reduced cost of a partial path $c(L)$,
- The can-collect $p(L)$ and can-deliver quantity $d(L)$,
- The feasible earliest $e(L)$ and latest departure time $\ell(L)$ at node $n(L)$,
- The total driving time $m(L)$ and route duration $o(L)$ upon arrival at node $n(L)$.

Here, $e(L)$ and $\ell(L)$ constitute a departure time interval for each partial path to accommodate dynamically choosing when to depart (and thus to arrive back) at a depot. It is suboptimal to wait at any point in the route except when arriving too early at a customer, so any route in the departure time interval does not contain any unnecessary waiting time. Note that $e(L)$ is feasible for all customer release times in the route, while $\ell(L)$ might not be feasible for customer deadlines. Furthermore, we store all customers in the partial path to ensure the elementarity of a route, and we use the non-increasing concepts of can-collect and can-deliver from Dell'Amico et al. (2006), which indicate the amount that can still be collected and can still be delivered by extending the label. The initial (empty) label is defined as $L_0 = ((), -\eta_i^k, Q^k, Q^k, 0, T, 0, 0)$. Recall that η_i^k is the dual cost associated with constraint (34), Q^k is the vehicle capacity of vehicle type $k \in K$, and T is the length of the time horizon.

4.3. Label feasibility

When we extend label L into a label L' with customer $j \in N_c$, and associated request $r(j)$, along arc $(n(L), j) \in \mathcal{A}$, the extension is feasible only if

$$j \notin \vec{N}(L), \tag{39}$$

$$d(L) - q^{r(j)} \geq 0 \qquad \text{if } j \in N_d, \tag{40}$$

$$p(L) - q^{r(j)} \geq 0 \qquad \text{if } j \in N_p, \tag{41}$$

$$p(L) \geq Q^k - \beta^k \qquad \text{if } j \in N_d, \tag{42}$$

$$e(L) + \tau_{n(L),j}^k + p_j \leq \ell_j, \tag{43}$$

$$b^{r(j)} + o(L) + \tau_{n(L),j}^k + p_j \leq \ell_j \qquad \text{if } j \in N_d, \tag{44}$$

$$\vec{\Psi}(L') \neq \emptyset. \tag{45}$$

Here, $\vec{\Psi}(L')$ represents the set of all feasible customer-linehaul assignments in the extended label. As discussed in Section 4.5, this refers to determining if a departure time exists at the depot for which all visited customers can be assigned to a linehaul if necessary. Condition (39) ensures elementarity of the label, conditions (40) and (41) ensure the vehicle has enough capacity, and condition (42) ensures the fill-up-to quantity is enforced. Moreover, conditions (43) and (44) ensure the customer can be served before the end of its time window. Finally, condition (45) forbids extensions that lead to infeasible customer-linehaul assignments and thus considers the feasibility of the customer release time, deadline, and the time synchronization between vehicle and linehaul.

4.4. Label extension

If the extension of label L with customer $j \in N_c$, and associated request $r(j)$, over arc $(n(L), j)$ is feasible, we create a new label L' as follows:

$$\vec{N}(L') = (\vec{N}(L), j), \tag{46}$$

$$c(L') = c(L) + \hat{\delta}^k_{n(L),j} + c(\psi^*(L')) - c(\psi^*(L)), \tag{47}$$

$$p(L') = \begin{cases} p(L) & \text{if } j \in N_d \\ p(L) - q^{r(j)} & \text{if } j \in N_p, \end{cases} \tag{48}$$

$$d(L') = \begin{cases} d(L) - q^{r(j)} & \text{if } j \in N_d \\ \min\{d(L), p(L) - q^{r(j)}\} & \text{if } p(L) \neq Q^k \text{ and } j \in N_p \\ \min\{d(L), p(L) - q^{r(j)}, \alpha^k\} & \text{if } p(L) = Q^k \text{ and } j \in N_p, \end{cases} \tag{49}$$

$$e(L') = \begin{cases} \max\{e(L) + \tau^k_{n(L),j},\ e_j,\ b^{r(j)} \\ +o(L) + \tau^k_{n(L),j}\} + p_j & \text{if } j \in N_d \text{ and } \mu^{r(j)} = i \\ \max\{e(L) + \tau^k_{n(L),j}, e_j\} + p_j & \text{if } j \notin N_d \text{ or } \mu^{r(j)} \neq i, \end{cases} \tag{50}$$

$$\ell(L') = \begin{cases} e(L') & \text{if } e(L) = \ell(L) \text{ or } \ell(L) + \tau^k_{n(L),j} \leq e_j + p_j \\ \min\{\ell(L) + \tau^k_{n(L),j} & \\ +p_j, \ell_j\} & \text{if } e(L) \neq \ell(L) \text{ and } \ell(L) + \tau^k_{n(L),j} > e_j + p_j, \end{cases} \tag{51}$$

$$m(L') = m(L) + \tau^k_{n(L),j}, \tag{52}$$

$$o(L') = o(L) + \tau^k_{n(L),j} + p_j + \max\{0, e_j - \ell(L) - \tau^k_{n(L),j}\}. \tag{53}$$

where $c(\psi^*(L'))$ and $c(\psi^*(L))$ in Eq. (47) represent the reduced cost of the optimal linehaul assignment solutions for labels L' and L, respectively. Here $\psi(L) \in \Psi(L)$. The next subsection discusses finding the optimal linehaul assignment solution for a label. Eqs. (48) and (49) generalize the can-collect and can-delivery quantity from Dell'Amico et al. (2006) and extend it by enforcing the mixing quantity α^k by reducing the can-delivery quantity to α^k when we perform the first collection in the route. Eq. (50) sets the earliest departure time of label L' based on the maximum of the earliest arrival time at the node j and the opening time of the customer time window. We directly account for a possible later departure if the order release time of an order originating from depot i forces us to do so. That is, we set $e(L')$ to $b^{r(j)} + o(L) + \tau^k_{n(L),j}$ if required. Eq. (51) sets the latest departure time of label L' based on the minimum arrival time at the node j and the customer's closing time window. If we wait earlier in the route or at customer j, the latest departure time becomes equal to the earliest departure time, and the length of the departure time interval becomes zero. Eq. (53) extends the duration of the route where $\max\{0, e_j - \ell(L) - \tau^k_{n(L),j}\}$ captures the waiting time at customer j. Note that the departure time interval length $\ell(L) - e(L)$ is non-increasing for any extension.

4.5. The customer-linehaul assignment problem

To find the reduced cost of the extended label, we need to determine the optimal customer-linehaul assignment $\psi^*(L)$ of a label. The goal is to find a route start and end time and to assign each visited customer not associated with depot i to a linehaul $s \in S$. For the assignment of customer $j \in \vec{N}(L)$ to linehauls $s \in S$, we incur cost $q^{r(j)}(f_s - \phi_s)$. The optimal customer-linehaul assignment asks for determining the route's start time t_{START} and end time t_{END}. The following four conditions should hold for a feasible linehaul assignment:

1. The route must depart after the release time of any delivery customer visited by the route. So,

$$t_{\text{START}} \geq b^{r(j)} \quad \forall j \in \{\vec{N}(L) \cap N_d\}. \tag{54}$$

2. The route must arrive before the deadline of any pickup customer visited by the route. So,

$$t_{\text{END}} \leq o^{r(j)} \quad \forall j \in \{\vec{N}(L) \cap N_p\}. \tag{55}$$

3. Each delivery customer $j \in N_d$ in a route associated with another depot $\mu^{r(j)} \neq i$ is assigned to a linehaul on arc $(\mu^{r(j)}, i)$ that departs after the delivery is released and arrives before the route departs from depot i. So,

$$\sum_{s \in S(\mu^{r(j)}, i) | d_s \geq b^{r(j)}, a_s \leq t_{\text{START}}} \gamma^u_{js} = 1 \quad \forall j \in \{\vec{N}(L) \cap N_d | \mu^{r(j)} \neq i\}. \tag{56}$$

4. Each pickup customer $j \in N_p$ in a route associated with another depot $\mu^{r(j)} \neq i$ is assigned to a linehaul on arc $(i, \mu^{r(j)})$ that departs after the route arrives at depot i and arrives before the deadline of the pickup customer. So,

$$\sum_{s \in S(i, \mu^{r(j)}) | d_s \geq t_{\text{END}}, a_s \leq o^{r(j)}} \gamma^u_{js} = 1 \quad \forall j \in \{\vec{N}(L) \cap N_p | \mu^{r(j)} \neq i\}. \tag{57}$$

Where the set $S(i_1, i_2) \in E$ in conditions (56) and (57) contains the subset of linehauls that drive depot arc (i_1, i_2). As these conditions show, the start and end time of the route determine which linehauls are feasible. We therefore consider multiple start times to find an optimal one, where a start time directly defines the end time by $t_{\text{END}} = t_{\text{START}} + o(L) + \tau^k_{n(L),i}$. The departure time t_{START} is bounded by $[e(L) - o(L), \ell(L) - o(L)]$ since departing earlier would result in redundant waiting, and departing later is infeasible. All customer time windows are respected within this interval. We define an *extreme* route start time as a start time that may lead to an optimal linehaul assignment. These extreme route start times are given by $\{a_s \mid s \in S^-_{r(j)}(i), e(L) - o(L) \leq a_s \leq \ell(L) - o(L)\} \cup \{e(L) - o(L)\}$, i.e., all arrival times of linehauls at depot i and the earliest feasible start $e(L) - o(L)$. We check if each extreme departure time can serve as a starting time. We assign customers to linehauls based on the feasibility conditions defined earlier. If any customer that requires a linehaul cannot be assigned to a linehaul for the current extreme departure time, the extreme departure time is infeasible. The current label extension is infeasible if all extreme departure times are infeasible. When there are multiple feasible extreme departure times, we select the extreme departure time with minimal reduced cost. Finally, we consider the reduced cost of the solution in extended label L' as defined in Eq. (47).

4.6. Dominance criterion

We use the following dominance criterion in our labeling algorithm: a label L_1 dominates a label L_2 if

$$n(L_1) = n(L_2), \quad \vec{N}(L_1) \subseteq \vec{N}(L_2), \quad c(L_1) \leq c(L_2), \quad p(L_1) \geq p(L_2),$$
$$d(L_1) \geq d(L_2), \quad \ell(L_1) - o(L_1) \geq \ell(L_2) - o(L_2), \quad e(L_1) \leq e(L_2),$$
$$m(L_1) \leq m(L_2), \quad o(L_1) \leq o(L_2),$$

and at least one of these inequalities is strict.

These inequalities ensure that any feasible extension for label L_2 is also feasible for label L_1, and the resource consumption in any extension of label L_2 is larger than or equal to the corresponding resource consumption in label L_1. The dominance conditions ensure that customers in L_1 can be assigned the same or more linehauls than in L_2 due to $\ell(L_1) - o(L_1) \geq \ell(L_2) - o(L_2)$ and $e(L_1) \leq e(L_2)$.

5. Adaptive large neighborhood search

This section presents an adaptive large neighborhood search (ALNS) for the VRPPDL. ALNS is often applied to pickup-and-delivery vehicle routing problems and can handle complex constraints, such as mixed levels and linehaul assignments, while efficiently exploring the solution space. Compared to the classical ALNS proposed by Ropke and

Pisinger (2006), our operators also consider linehaul assignments when destroying and repairing a solution. Moreover, we accelerate large-scale instance solving by integrating an efficient route evaluation strategy that leverages spatiotemporal locality, and we speed up computation by parallelizing the ALNS.

We first introduce the algorithm's initialization, destroy, and repair stages. Then, we describe the simulated annealing acceptance criterion and the adaptive operator weight adjustment of the algorithm. Subsequently, we discuss how the ALNS is applied to populate the route set of the restricted master problem by solving the ESPPRTSC. Lastly, we discuss acceleration techniques for the algorithm.

5.1. Initialization stage

The ALNS constructs an initial solution using n_{INIT} attempts of a greedy insertion operator with random noise. This operator, described in Section 5.3, ensures the feasibility of the solution. After n_{INIT} attempts, the ALNS initializes by storing the best solution X^b as the current solution X^c. The fleet size constraint is relaxed to ensure an initial solution is found when the number of available vehicles is tight. Still, solutions that use excessive vehicles incur a large penalty. The probability of the destroy and repair operators is uniformly assigned. After initialization, the ALNS tries to improve its current solution by moving to a neighbor solution, which we denote as the candidate solution X^t.

5.2. Destroy stage

The destroy stage removes a subset of customers from the candidate solution X^t, where the number of customers removed n_{REM} is selected using a uniform random distribution $[n^-_{\text{REM}}, n^+_{\text{REM}}]$. Here, n^-_{REM} and n^+_{REM} are the minimum and maximum number of customers to remove, respectively. When removing customers, we also remove their associated linehaul assignments. The vehicle capacity and mixing level constraints are maintained during removal.

We always service each customer early to preserve the feasibility of the time window. This means that when a delivery customer with the latest release time is removed from the route, the route's departure time is adjusted accordingly. An example is shown in Fig. 2. Bringing the route departure time forward may increase the route duration, rendering the total route duration constraint infeasible. In this case, the entire route is removed from the candidate solution. These routes can be feasible if we depart later, but they are unlikely to be part of an optimal solution due to their extensive waiting times.

The destroy stage uses six removal operators:

1. *Dynamic removal*: The operator starts at a random customer, which we denote as the anchor node. From this anchor node, it moves in both directions along the route to remove $\tilde{\kappa} n_{\text{REM}}$ customers. Here, $\tilde{\kappa} \in [0, 1]$, which we set to 0.5, controls the behavior of the operator. If $\tilde{\kappa} n_{\text{REM}}$ is larger than the full length of the route, the entire route is removed. Then, the operator removes the nearest customers from the anchor node in the solution until n_{REM} customers are removed. Fig. 3 shows a visual example of this operator.
2. *Random route removal*: This operator removes routes randomly until at least n_{REM} customers are removed.
3. *Greedy route removal (with noise)*: This operator calculates the average cost per node for each route and removes routes with the highest costs per node until at least n_{REM} customers are removed. The cost per node is the route's total cost divided by the number of customers served. This operator is also added with noise level ϖ, where we multiply the cost per node with a random number between $[1 - \varpi, 1 + \varpi]$.
4. *Excessive route removal*: This operator removes routes of the vehicle type(s) currently penalized for using more than the allowed amount of vehicles until at least n_{REM} customers are removed. If no vehicle types are penalized, this operator randomly selects customers to remove.
5. *Waiting time route removal*: This operator iteratively removes the route with the highest waiting time, defined as the route duration minus the route driving time and the total service time, until at least n_{REM} customers are removed.

5.3. Repair stage

The repair stage inserts all removed customers back into the candidate solution. The algorithm finds a route position, linehaul assignment, and cost for each customer insertion. When we insert a delivery request in a route from a different depot, we assign the request to the earliest feasible linehaul between the depots. Similarly, if we insert a pickup request in a route from a different depot, we assign it to the latest feasible linehaul between depots. When no linehaul with sufficient capacity is available between the origin depot of a request and another depot, all routes from the other depot are no longer considered. While inserting, we also check if opening a new route is cheaper than inserting the request into existing routes.

The same principles discussed in Section 5.2 are applied during insertion to maintain time window feasibility. This means the inserted customer is served as soon as possible. Furthermore, a customer can only be inserted into a route that respects its release time or deadline.

The repair stage uses two operators:

1. *Greedy insertion (with noise)*: This operator inserts all customers at the feasible position with the lowest insertion cost, including linehaul costs. This operator is added with noise level ϖ, where we multiply the insertion cost at each feasible position with a random number between $[1 - \varpi, 1 + \varpi]$.
2. *First feasible insertion*: This operator goes over all routes in the solution in random order and inserts a customer in the first route with a feasible insertion position. If multiple feasible insertion positions are possible on the route, the position with the lowest detour length is selected.

5.4. Acceptance criteria and probability adjustment

We use the Pulsed Threshold Acceptance (PTA) criteria, as introduced by Bustos-Coral and Costa (2025), that builds upon the Record-to-Record Travel (RRT) acceptance criteria proposed by Santini, Ropke, and Hvattum (2018). RRT evaluates a candidate solution based on the gap between the candidate and best solution, instead of the gap between the candidate and current solution. This allows the search to escape local optima over multiple iterations. The gap is reduced linearly from an initial value A_{THRES} to zero over time. PTA extends RRT by introducing a pulsed threshold mechanism. Specifically, it uses a second threshold that decreases to zero in A_{N} iterations, encouraging improvements early on in the search. If no new best solution is found in A_{N} iterations, and the second threshold reaches zero, the current solution is set to the best solution found so far. The second threshold is set to A_{THRES}, allowing the search to explore again. Also, if a new best solution is found, the second threshold is immediately reset to A_{THRES}.

We apply the probability weight adjustment introduced by Ropke and Pisinger (2006). The weight of each destroy and repair operator w_ρ is updated based on the improvement of the candidate solution X^t and the operators used in this iteration. Given that operator ρ is used this iteration, its weight w_ρ is updated using

$$w_\rho = w_\rho(1 - \xi) + \Phi_\rho \xi, \tag{58}$$

where Φ_ρ is the score of operator ρ and ξ controls the adaptability of the algorithm. This score is three when the candidate solution X^t is a global best, two when the candidate solution X^t is better than the current solution X^c, one when it is accepted, and zero otherwise. Afterward, the weight vectors of the destroy and repair operators are normalized.

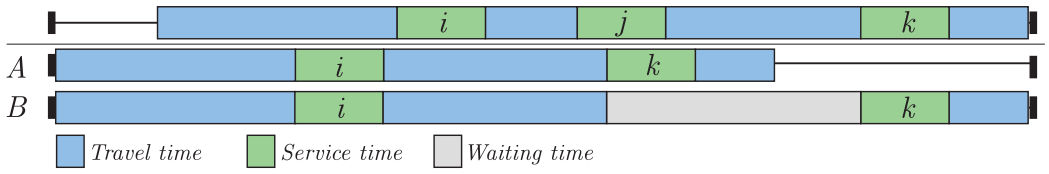


Fig. 2. Scenarios for the removal of a request.
Note. Delivery customer j has the latest order release time. After its removal, the departure time of the route is brought forward. First, the situation before the removal is shown. In scenario A, customers' time windows i and k allow them to be served consecutively. In scenario B, the vehicle waits until the customer k opens the time window.

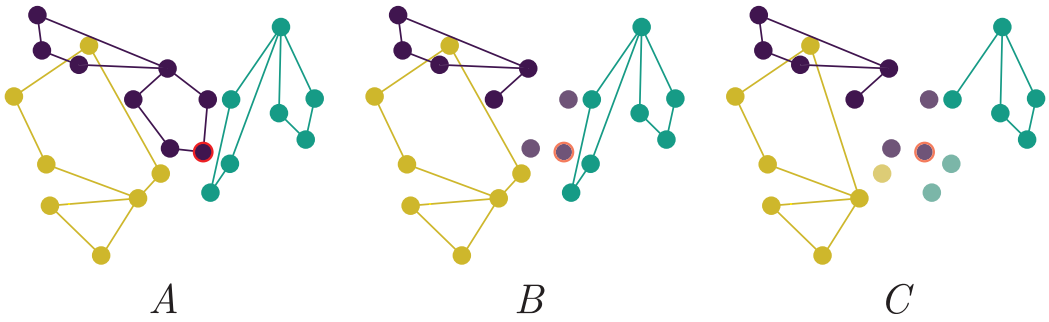


Fig. 3. Dynamic removal operator.
Note. Given an example solution, $\bar{\kappa} = 0.5$, and $n_{\text{REM}} = 6$. Step A selects a random anchor node, circled in red. Step B removes $\bar{\kappa} n_{\text{REM}} = 3$ customers along the route of the anchoring node. Finally, step C removes the nearest customers to the anchor node until the operator reaches a total of n_{REM} removed customers. Route colors indicate associated depots. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

5.5. Integrating the alns in the restricted master heuristic

The ALNS is integrated into two parts of the restricted master heuristic. First, the heuristic finds an upper bound for the master problem and an initial set of routes for the restricted linear master problem (RLMP). This initial set of routes is filled with all unique routes from the n_{SOL} best solutions encountered during ALNS. Second, the ALNS finds negative reduced-cost routes by incorporating dual values into its objective. We call this the ALNS-assisted restricted master heuristic. Since the ALNS searches for the overall lowest-cost solution, rather than the lowest-cost individual route, we check all routes encountered during the repair stage for a negative reduced cost. We store the $n_{\text{COLS,ALNS}}$ routes with the lowest negative reduced cost and add them to the RLMP once the ALNS terminates. This ensures we evaluate all routes, not just those accepted as the current or best solution. When the ALNS does not find new reduced-cost routes, the exact labeling algorithm, described in Section 4, is used to verify if the current RLMP solution is optimal. Suppose the exact labeling algorithm finds any reduced-cost routes. In that case, we solve the new RLMP and then fall back to the ALNS to check if additional routes are found before reapplying the exact labeling algorithm.

5.6. Acceleration techniques for large-scale instances

To speed up computation, the ALNS is parallelized. In the parallel setup, n_{THREAD} instances of the metaheuristic are executed in parallel. After a preset number of iterations n_{MAX}, the best solution among all threads is selected and redistributed across all threads. This process is repeated n_{STEP} times. Only the solution is shared between threads. Each thread runs on a different random seed, and the operator weights are maintained for each thread to diversify the search.

Furthermore, the search space for the insertion operators is decreased by utilizing the spatiotemporal locality of good solutions. When inserting a node, only the routes containing at least one of n_{LOC} nearest neighbors to this node are considered for insertion. For larger problem instances, this drastically reduces the number of routes considered for insertion without compromising solution quality significantly, as shown by Arnold, Gendreau, and Sörensen (2019).

6. Computational results

This section describes the computational experiments. First, we introduce and describe our set of publicly available VRPPDL benchmark instances, which are based on our industry partner's real-life data. Then, we introduce our various restricted master heuristic variants for solving the VRPPDL, each incorporating differing degrees of the ALNS. Subsequently, we compare the performance of the different variants. Lastly, we assess the ALNS's standalone performance.

The restricted master heuristic and ALNS are implemented in C++. Gurobi is used to solve (mixed-integer) linear programs. All small VRPPDL instances are solved using a 2.6 GHz Intel Core i7 CPU with 16 GB of RAM and six physical threads. Finally, reduced and large-scale VRPPDL and Homberger and Gehring (2005) instances are solved on a Linux server with a 2.4 GHz Intel Xeon CPU, 128 GB of RAM, and 16 physical threads.

6.1. Benchmark instances

In collaboration with our industry partner, we propose a set of realistic, large-scale instances for the VRPPDL. These instances are derived from our industry partner's real-life shipment data. We anonymize customer data by adding slight noise to their coordinates and demand data. We present 48 realistic large-scale instances, each containing data from a single day in 2023. We propose 24 instances from the Netherlands (NL) and 24 instances from Belgium (BE), which both consider 3 depots, but differ in depot locations, linehauls, and demand structure.

Besides the large-scale instances, 48 reduced instances are presented, where the customers are a random subset of the customers in the large-scale instance. In these instances, the number of available vehicles is reduced to match the decreased demand. The instances are named *[C]_[D]_[I]_[S]*, where *C* denotes the country, which is NL for the Netherlands and BE for Belgium, *D* denotes the weekday, *I* denotes the instance number ranging from 0–4, and *S* denotes the size of the instance, which is F for the large-scale instances and R for the reduced

Table 1
Performance of the restricted master heuristics.

		CG1			CG2		CG3		HCG1			ALNS	
$\|N_0\|$	$\|N_c\|$	LB	UB	Time (s)	UB	Time (s)	UB	Time (s)	RLMP	UB	Time (s)	UB	Time (s)
2	10	541.7	542.2	0.1	542.2	0.4	542.2	0.2	**541.7**	542.2	0.2	542.2	0.0
2	15	691.2	691.2	0.6	691.2	1.1	691.2	0.9	**691.2**	691.2	0.8	691.9	0.1
2	20	924.0	924.0	3.4	924.0	3.2	924.0	2.1	**924.0**	924.0	1.7	924.0	0.2
2	25	1055.5	1063.5	4.0	1063.5	5.2	1063.5	3.2	1055.6	1067.5	2.5	1075.2	0.3
2	30	958.0	964.4	34.7	964.7	29.9	964.7	6.9	**958.0**	964.7	3.3	964.4	0.5
2	35	1174.6	1229.4	753.3[1]	1210.9	483.1[1]	1187.1	154.6	1175.0	1187.1	4.6	1206.9	0.8
2	40	1377.3	1408.4	313.8	1403.6	408.8	1402.8	215.1	1378.4	1402.8	5.9	1416.9	1.2
2	45	TLR	1645.2	675.4[1]	1656.7	825.9[1]	1522.1	15.3[1]	1519.7	1522.1	7.0	1551.9	1.5
2	50	TLR	2331.9	TLR[3]	2321.1	TLR[3]	1534.2	1948.7[1]	1499.8	1534.2	10.0	1549.5	2.3
2	55	TLR	1818.9	TLR[3]	2134.3	TLR[3]	1565.4	73.8[1]	1552.7	1565.4	10.4	1573.9	2.9
2	60	TLR	2353.1	TLR[3]	2355.3	TLR[3]	1733.6	342.3[2]	1721.8	1733.6	12.9	1733.7	4.0
2	65	TLR	2895.3	TLR[3]	2822.5	TLR[3]	1900.4	TLR[3]	1851.6	1900.4	14.7	1884.8	5.0
2	70	TLR	3301.3	TLR[3]	3333.7	TLR[3]	1945.2	TLR[3]	1876.7	1945.2	17.3	1928.0	6.2
2	75	TLR	3524.1	TLR[3]	3423.4	TLR[3]	1971.6	2088.2[2]	1942.4	1971.6	18.5	1986.4	7.6
2	100	TLR	4569.8	TLR[3]	4425.8	TLR[3]	2237.1	TLR[3]	2198.2	2242.5	29.2	2233.7	18.8
2	125	TLR	7364.4	TLR[3]	6521.3	TLR[3]	3055.9	TLR[3]	2912.6	3055.9	66.2	2968.9	37.1
2	150	TLR	TLR	TLR[3]	7648.4	TLR[3]	3517.5	TLR[3]	3218.4	3517.5	257.5	3205.1	63.1

instances. All instances with transcripts of the reported solutions are available in our GitHub repository.[3]

As the VRPPDL is a new problem, no benchmark instances are available in the literature. To benchmark the performance of the ALNS algorithm, the CVRPTW benchmark instances presented by Homberger and Gehring (2005) are converted into VRPPDL instances. The CVRPTW is a special case of the VRPPDL with a single depot, a single vehicle type, no linehauls, only delivery (or pickup) requests, and no depot time windows. We use the CVRPTW instances with 200 and 1000 customers. Sixty instances are available for both instance sizes, spread evenly across six different distributions. We compare our solutions to the best-known distance-only objective solutions for the benchmark instances reported in CVRPLIB (see, e.g., Uchoa et al., 2017). We want to emphasize that our algorithms are designed for a more complex real-life vehicle routing problem, and therefore, the comparison with the CVRPTW is solely for validation purposes.

The realistic VRPPDL instances are too large to be solved using the exact labeling algorithms. Therefore, a set of small VRPPDL instances is generated using real-world data distributions. Three instances are generated for each problem size, ranging from 10 to 75 customers in increments of 5, and from 75 to 150 customers in increments of 25. These instances consider 2 depots and are named *P_[|N_c|]_[Z]*, where $|N_c|$ denotes the number of customers and Z denotes the instance number ranging from 0–2.

6.2. Restricted master heuristic implementations

We consider multiple restricted master heuristics to study the effect of ALNS-assisted variants. The first restricted master heuristic (CG1) employs the exact labeling algorithm. The second restricted master heuristic (CG2) also employs the exact labeling algorithm, but the route set is initialized using high-quality ALNS solutions. The third restricted master heuristic (CG3) initializes the route set and finds reduced cost routes heuristically using the ALNS. The exact labeling algorithm is employed once the ALNS cannot find any new reduced-cost routes. Finally, the fourth restricted master heuristic (HCG1) is similar to CG3, but does not employ the exact labeling algorithm and thus cannot guarantee root-node optimality.

The ALNS parameters are tuned using a sequential search procedure. More details on the tuning procedure and the used parameters are provided in Appendix A. We set n_{SOL} to 100 and $n_{\text{COL,ALNS}}$ to 200.

[3] github.com/siemterbraake/vrppdl-algorithms-instances

6.3. Performance of the restricted master heuristic variants

The performance of the restricted master heuristics is tested using small-scale problem instances. If an exact lower bound is found, it is reported. Furthermore, the time to find a lower bound is reported. Also, we report the upper bound obtained using the route set after solving the restricted master problem (RMP). Since HCG1 does not guarantee lower bounds, we report the best relaxed linear master problem (RLMP) solution found, as well as its runtime. If the lower bound of HCG1 is optimal, it is highlighted in bold. All three instances are solved for each instance size, and the results are aggregated in Table 1. In the table, TLR indicates that the limit of 3600s is reached for all instances, and the superscript denotes the number of instances at which the algorithm timed out. When only a selection of the instances is solved, the average times only include the instances solved by all algorithms. The individual results for all instances are reported in Appendix B.

The CG1 algorithm solves instances of up to 45 customers but reaches the time limit for one of the 35-customer instances and one of the 45-customer instances. We observe that the time limit is reached for 50-customer instances and larger, and hence no lower bounds can be provided anymore. The CG2 algorithm solves the same instances as the CG1 algorithm. The CG3 algorithm solves all instances solved by the CG1 and CG2 algorithms and finds a solution for the 35-customer instance that the CG1 and CG2 algorithms did not solve within the time limit. The HCG1 and ALNS algorithms find solutions for all instances.

ALNS initialization, as applied in CG2, does not decrease the time to find a lower bound compared to CG1, where no ALNS is used. However, the CG3 algorithm with ALNS initialization and ALNS route set population is more efficient than the other exact labeling algorithms for instances with more than fifteen customers. For the solved instances with 45 customers, the CG3 algorithm reduces computation time compared to CG1 by 98% while maintaining root-node optimality. However, for instances with 50 customers or more, the complexity of the exact labeling algorithm explodes, causing all restricted master heuristics that use it to reach their time limit.

The heuristic HCG1 algorithm finds a good approximation of the optimal root-node solution. The HCG1 algorithm finds RLMP solutions within 0.04% of the optimal root-node solution, as indicated in the LB column. Compared to the fastest restricted master heuristic that guarantees lower bounds, CG3, HCG1 reduces computation time by 69% for the 45-customer instances that CG3 solves. When comparing HCG1 with simply running the ALNS, we observe that the ALNS is competitive with HCG1. On average, the upper bounds from the ALNS are 0.2% lower than the upper bounds from HCG1. Furthermore, the ALNS is more efficient in solving larger instances in a significantly

Table 2
Performance of the restricted master heuristics for instances with more depots.

			CG1		CG2		CG3		HCG1			ALNS	
$\|N_0\|$	$\|N_c\|$	LB	UB	Time (s)	UB	Time (s)	UB	Time (s)	RLMP	UB	Time (s)	UB	Time (s)
4	40	1606.2	1616.5	78.6	1616.2	90.1	1616.9	24.9	1606.5	1622.9	6.5	1622.6	1.5
4	50	1951.3	1982.1	1630.5[1]	1962.5	2355.2	1961.8	163.9	1955.0	1961.8	10.1	1963.3	2.9
6	40	1820.6	1952.5	177.8[1]	1829.3	1091.3	1832.4	13.4	1820.7	1832.4	7.4	1830.8	2.0
6	50	2133.8	2137.6	244.9	2137.6	292.8	2137.8	44.0	2134.4	2137.8	10.8	2174.0	3.6

Table 3
Summarized results on 200 customer CVRPTW benchmark instances.

Instance	BKS	Best ALNS solution	Avg ALNS solution	Time (s)	Gap (%)
c1	**2672.7**	2679.0	2679.9	61.3	0.23
c2	**1820.8**	1830.1	1833.2	107.1	0.51
r1	**3572.9**	3601.1	3617.5	51.1	0.80
r2	**2621.9**	2642.3	2657.2	108.2	0.76
rc1	**3151.3**	3179.3	3194.3	46.1	0.91
rc2	**2286.3**	2300.0	2306.7	113.7	0.59
Overall	**2687.7**	2705.3	2714.8	81.3	0.63

shorter time. Therefore, the ALNS is our method of choice for analyzing the large-scale VRPPDL instances in the remainder of this paper.

6.4. The impact of more depots on the performance of the restricted master heuristics

The performance of the restricted master heuristics is also tested on problem instances with more depots. The depots are evenly distributed over the geographical area. Table 2 follows the same format as Table 1, and detailed results for individual instances are given in Appendix B.

All instances with four and six depots are solved by the restricted master heuristics. For CG1 and CG2, increasing the number of depots leads to reduced computation times for instances with 40 customers, but results in longer computation times for instances with 50 customers. As the number of depots increases, the computation time of the CG3 algorithm decreases, even though more pricing problems need to be solved. Algorithms that do not use the exact labeling algorithm require more time to solve instances with more depots, due to the increased number of pricing problems.

6.5. Performance of the ALNS

We validate the standalone use of the ALNS by using the CVRPTW benchmark instances introduced in Section 6.1. We compare the results from our ALNS to the best-known solutions from the CVRPLIB on total distance. The best-known solutions (BKSs) from the CVRPLIB were found within a 2-hour time limit. Average BKSs obtained from an average of optimal solutions are highlighted in bold. Each instance is solved five times with a fixed number of algorithm iterations. The results are grouped by instance type, and the average results are reported. We report the best-known solution (BKS), the best solution found over five runs, the average solution found over five runs, the average computation time per run (in seconds), and the average gap between the BKS and the best solution found over five runs. Results for individual instances are reported in Appendix C, as are the results for the 1000-customer instances.

From Table 3, we observe that the ALNS finds solutions with an average gap of 0.63% with an average computation time of 81 seconds. This shows that the ALNS can find high-quality solutions. Additionally, the solution obtained over the five runs is, on average, only 0.34% higher than the best solution found for each instance. This indicates the algorithm is robust. The number of ALNS iterations is kept low, resulting in a low computation time. With more ALNS iterations, the quality of the solutions can be further improved.

We then focus on solving the real-life benchmark instances, using the ALNS to provide solutions. Each instance is solved five times. For each instance, we report the size $|N_c|$, the best solution over five runs, the average solution over five runs, the average computation time in seconds, the number of vehicles used $|V|$, the average number of customers per vehicle $|N_c| / |V|$, and the average linehaul utilization. The results are grouped by country and weekday. The results for the reduced instances are reported in Appendix C, and the results for the large-scale instances are reported in Table 4.

The large-scale instances in Table 4, with an average of 1602 customers, are solved in 853.3 seconds on average. The vehicles can serve multiple orders in a single route, with an average of 20.4 customers served per route. The linehaul utilization varies heavily between instances, ranging between 36.8–97.0%. The linehaul utilization increases as the number of customers increases, which makes sense. Furthermore, the linehaul utilization in Belgium is consistently lower than in the Netherlands. No significant differences are observed between weekdays. All instances and detailed individual solutions are provided in our GitHub repository.

7. Case study at a European logistics service provider

This section discusses how we employed our ALNS algorithm to solve real-world problem instances for our industry partner, a major European logistics provider. The first subsection discusses the data processing steps. The second subsection compares the solutions obtained using ALNS to the actual schedules at our logistics partner. In the last section, we analyzed the cost associated with geographical service area restrictions for depots.

7.1. Data processing

For this case study, 24 instances representing customer requests on 24 different weekdays spread evenly throughout 2023 are constructed. This case study utilizes a real dataset without noise and features a realistic road distance matrix derived from travel time calculation software provided by our industry partner. The release time and deadlines for requests are based on their international linehauls. Small requests at the same approximate location, with the same time windows and international linehauls, are combined to reduce the instances' size. Requests are only combined if the combined quantity of the requests is below the smallest vehicle's capacity. The instances use the same unit of quantity as our industry partner, and each customer's service time is set to 14 min, which is the average service time of our industry partner.

Our logistics partner uses four types of vehicles to service their customers. The maximum number of vehicles available at each depot is set for each vehicle type to the maximum number used in the 24 days

Table 4
Results real-life VRPPDL instances.

Country	Weekday	Id	$\|N_0\|$	$\|N_c\|$	Best 5	Avg 5	Time (s)	$\|V\|$	$\|N_c\|/\|V\|$	Linehaul util. (%)
NL	MO	0	3	1694	28 008.4	28 798.9	867.4	76	22.3	76.00
		1	3	2047	32 690.8	32 996.4	992.0	89	23.1	88.80
		2	3	1915	29 753.2	29 977.0	917.5	85	22.5	85.00
		3	3	2052	33 083.5	33 410.9	926.3	92	22.4	91.80
		4	3	2082	32 846.2	33 278.3	987.1	89	23.5	88.60
NL	TU	0	3	2039	30 932.0	31 321.9	962.8	88	23.3	87.60
		1	3	2218	34 901.9	35 320.9	970.4	93	23.8	93.00
		2	3	2108	31 834.8	32 303.7	947.3	91	23.1	91.40
		3	3	2040	31 086.4	31 455.5	938.3	84	24.2	84.40
		4	3	1958	30 622.0	30 905.4	888.5	86	22.8	86.00
NL	WE	0	3	2040	33 194.8	33 543.4	941.7	89	23.0	88.60
		1	3	2237	35 092.6	35 308.4	977.2	97	23.1	97.00
		2	3	2143	33 487.2	33 761.6	1014.7	92	23.3	91.80
		3	3	1388	23 838.9	23 991.1	780.1	66	21.0	66.20
		4	3	2021	32 694.1	33 266.6	913.1	90	22.5	90.00
NL	TH	0	3	2216	34 763.2	34 919.9	1021.7	95	23.3	95.20
		1	3	2062	32 283.7	32 435.2	966.0	89	23.3	88.60
		2	3	1810	28 950.2	29 269.3	881.9	79	22.9	79.00
		3	3	1909	31 100.0	31 522.0	892.9	85	22.6	84.60
NL	FR	0	3	2160	33 973.6	34 130.9	1014.8	91	23.6	91.40
		1	3	2139	33 163.4	33 532.0	990.6	91	23.5	91.20
		2	3	1799	29 294.3	29 608.7	877.1	80	22.5	79.80
		3	3	2056	32 443.4	32 810.2	960.8	91	22.5	91.20
		4	3	1301	21 923.3	22 364.0	691.5	66	19.7	66.20
BE	MO	0	3	1036	17 624.0	17 764.6	701.9	61	17.1	60.60
		1	3	1302	22 684.3	22 964.9	803.3	72	18.0	72.20
		2	3	1164	17 889.4	18 164.2	805.7	57	20.6	56.60
		3	3	1241	21 411.6	21 521.3	823.4	65	19.2	64.80
		4	3	1221	20 769.0	20 941.3	805.0	66	18.4	66.40
BE	TU	0	3	1208	19 620.8	19 808.4	773.7	67	18.0	67.00
		1	3	1313	22 155.1	22 506.9	786.4	74	17.8	73.80
		2	3	1192	18 679.3	19 024.3	765.6	63	19.0	62.60
		3	3	1395	22 295.2	22 619.8	873.6	71	19.6	71.00
		4	3	1282	21 216.8	21 591.2	771.5	78	16.4	78.00
BE	WE	0	3	1272	20 555.9	20 814.7	771.1	69	18.4	69.20
		1	3	1355	20 488.7	20 727.2	832.9	71	19.1	70.80
		2	3	1241	19 870.7	20 013.0	767.5	77	16.1	77.00
		3	3	1118	20 766.3	21 040.0	757.7	70	15.9	70.20
		4	3	1324	20 369.0	20 999.9	762.4	79	16.8	78.60
BE	TH	0	3	1250	19 438.8	19 942.6	767.1	75	16.7	75.00
		1	3	1274	20 289.0	20 424.6	780.5	72	17.7	71.80
		2	3	1126	17 595.2	17 838.8	695.6	64	17.6	64.00
		3	3	1260	20 523.9	20 815.9	763.2	72	17.5	72.00
BE	FR	0	3	1323	20 576.4	20 734.3	816.2	74	18.0	73.60
		1	3	1322	20 622.8	21 081.3	804.6	70	18.8	70.40
		2	3	1171	19 712.2	19 830.6	750.5	70	16.8	69.80
		3	3	1325	22 379.4	22 861.3	831.3	69	19.1	69.40
		4	3	737	12 117.1	12 214.0	627.0	37	20.0	36.80

considered. The costs per kilometer for each vehicle are determined by averaging the data over the 24 considered days for each vehicle type. Moreover, a non-negative fixed cost is set for all vehicle types to prevent unnecessary vehicles from being used. The vehicle costs are given in Table 5, and the number of available vehicles at each depot is given in Table 6. Following European driving regulations, the maximum driving and working times for all vehicle types are set at 9 and 13 h, respectively.

Our industry partner's weekly linehaul planning is taken into consideration in these instances. We assume that the drive time of linehauls is deterministic and set the cost of assigning a request to a linehaul based on the total cost of the linehaul divided by the capacity of the linehaul. All depots are connected by at least one linehaul at the start and the end of each day.

A realistic distance and time matrix was constructed using a third-party data provider for each instance and vehicle type. The third-party data provider considers the speed and routing limitations of each vehicle type.

7.2. Cost savings using the ALNS compared to practice

We use the parallelized ALNS algorithm to find case-study solutions. These solutions are compared to the realized costs for the corresponding day in Table 7; thus, each entry in the table represents the percentage difference between the executed solutions and those provided by our industry partner. We show the gap in the number of vehicles used $|V|$, the gap in driven distance, the cost gap, and the computation time in seconds.

A few observations stand out. First, the case-study solutions have an average cost gap of 17.4% compared to realized schedules. We recognize that the schedules provided by our industry partner are subject to practical disadvantages, such as last-minute shipment changes and traffic delays, which can widen the gaps. However, the gaps are sufficiently high to assume that the ALNS finds more efficient schedules than the current methods used by our industry partner. Second, the decrease in the number of vehicles used varies across instances and

Table 5
Fixed and variable vehicle costs.

Vehicle type	Netherlands		Belgium	
	Fixed	Variable	Fixed	Variable
Van	20	1.363	20	1.570
Box truck	20	2.091	20	2.416
City trailer	20	2.213	20	2.290
Trailer	20	1.965	20	2.651

Table 6
Number of available vehicles.

Vehicle type	Netherlands			Belgium		
	Depot 1	Depot 2	Depot 3	Depot 1	Depot 2	Depot 3
Van	12	8	5	13	7	4
Box truck	4	16	2	28	10	13
City trailer	26	21	14	6	1	6
Trailer	19	6	1	8	22	15

Table 7
Comparison of case-study solutions and realized schedules.

Instance	Gap (%) $\|V\|$	Gap (%) distance	Gap (%) cost	Time (s)
NL_1	−21.98	−33.68	−30.24	580.2
NL_2	−13.46	−22.30	−22.35	654.7
NL_3	−1.98	−14.19	−15.45	688.4
NL_4	−6.42	−14.44	−12.50	796.5
NL_5	−4.55	−13.13	−10.36	730.6
NL_6	−2.78	−15.76	−13.66	682.9
NL_7	−3.88	−14.01	−12.79	685.1
NL_8	−8.91	−17.11	−14.02	758.9
NL_9	−2.83	−17.69	−13.98	681.6
NL_10	0.00	−17.11	−15.32	772.5
NL_11	−11.34	−24.53	−24.91	700.9
NL_12	−1.87	−11.54	−10.62	727.7
NL_13	−3.85	−17.82	−16.78	685.9
NL_14	1.08	−16.35	−13.86	608.9
NL_15	−8.45	−23.02	−22.97	516.1
NL_16	−3.45	−17.03	−16.23	653.1
NL_17	−2.11	−14.16	−11.26	666.3
NL_18	−10.78	−24.23	−19.86	759.2
NL_19	−3.85	−13.64	−12.94	679.1
NL_20	−6.93	−22.71	−20.53	700.2
NL_21	−6.48	−23.52	−19.72	736.5
NL_22	−12.75	−23.51	−19.72	704.2
NL_23	−5.10	−19.18	−17.02	705.2
NL_24	−17.11	−20.68	−41.55	505.8
Overall	**−6.44**	**−18.61**	**−17.41**	**682.5**

appears to correlate with the obtained distance and cost savings. The different sizes of each instance cause the variation, but even when the same vehicle fleet size is used (see instance NL_10), we still obtain a 15.3% cost saving. Third, the provided solutions are obtained in an average computation time of 682.5 s, which is negligible considering the current planning process at our industry partner, where next-day planning is created overnight.

7.3. Impact of depot service areas restrictions

Major logistics service providers often allocate fixed service areas to their depots, where each depot solely processes all requests within its service area. This divides their operations into smaller problems per depot but restricts the distribution of requests among depots. Our industry partner does not strictly enforce their service area scenario, meaning that, in practice, human planners ignore service area restrictions to optimize their route planning in certain scenarios. Nevertheless, we aim to quantify the impact of service area restrictions on the total operational costs of local depots. We compare two scenarios: one with and one without fixed service areas. We incorporate service area restrictions into the ALNS by assigning requests to linehauls based on their respective service areas.

In the scenario with fixed service areas, forcing the assignment of shipments to linehauls based on their geographical service area becomes infeasible in two cases. Either the capacity of the linehaul is insufficient, or the customer time windows cannot be met due to the forced linehaul assignment. To ensure feasibility, we relax linehaul capacity and remove shipments that violate time windows in the restricted service areas scenario from both scenarios. This is one reason our industry partners' current operational planning also does not strictly enforce the service areas.

We compare the cost of twelve randomly selected instances with and without service area restrictions in Table 8. We report the number of customers $|N_c|$, the gap in objective value between the cost with and without the service area restriction, and the percentage of requests that are removed f_{inf} due to infeasible time windows in the scenario with fixed service areas. Six instances are based on the Dutch (NL) logistical network, and six instances are based on the Belgian (BE) logistical network of our industry partner. We take the service areas currently used by our industry partner as input for the scenario with service areas.

On average, enforcing service areas increases costs by 4.9%. In the best case, the strict service area solutions are 2.6% more expensive than the flexible solutions, but in the worst case, the difference is 8.0%.

Table 8
Impact of service area restrictions.

Instance	$\|N_0\|$	$\|N_c\|$	Gap (%) cost	f_{inf}
NL_1	3	1548	5.0	1.0
NL_5	3	1914	2.6	0.9
NL_9	3	1853	3.5	0.5
NL_13	3	1754	4.5	1.0
NL_17	3	1692	8.0	0.9
NL_21	3	1866	3.4	0.4
BE_1	3	883	4.6	0.8
BE_5	3	1081	6.1	1.8
BE_9	3	1063	4.1	2.6
BE_13	3	971	6.7	2.0
BE_17	3	1037	3.4	2.8
BE_21	3	1063	6.9	1.1
Overall		**1394**	**4.9**	**1.3**

Furthermore, imposing service areas causes infeasible time windows for 1.3% of customers. These shipments are scheduled outside their original service area or rescheduled for the next day. The first approach can lead to impractical vehicle routes, and the latter decreases service levels. The solutions, for instance NL_1, with and without service area restrictions, are shown in Fig. 4.

In the flexible service area scenario, we observe that depot 0 and depot 2 serve the western part of the Netherlands. Furthermore, one vehicle from Depot 0 drives far into the northern part of the Netherlands, which is otherwise served by Depot 1. In the strict service area scenario, all vehicle routes are contained within the service area. Although we show that flexible service area scheduling leads to more efficient routing solutions, the improvement from flexible routing alone does not cover the improvements observed in Table 7. This observation underscores that our proposed approach is likely to generate better routing solutions than the methods currently used by our industry partner.

8. Conclusions

We introduced the Vehicle Routing Problem with Pickups, Deliveries, and Scheduled Linehauls (VRPPDL), which models the logistics network of a major logistics service provider. The VRPPDL aims to find a cost-minimizing set of routes and assignment of transportation requests to linehauls while considering various operational constraints, including vehicle and linehaul capacities, mixing levels, time synchronization, customer time windows, and the maximum vehicle driving and working times. We first modeled the VRPPDL as a compact mixed-integer linear program that commercial solvers can solve. Then, we decomposed this formulation by vehicle type and depot to arrive at a set-covering formulation, where each column represents a vehicle route and the assignment of its customers to linehauls. The subproblem is an elementary shortest path problem with resource and time synchronization constraints, for which we developed a new exact labeling algorithm to solve it optimally.

In addition to a restricted master heuristics that contain the exact labeling approach, we developed a tailored adaptive large neighborhood search (ALNS) algorithm for solving the VRPPDL. The ALNS serves three main purposes. First, we used it standalone to solve practically sized large-scale instances. Second, we used the solutions obtained from the ALNS as initial solutions for the column generation procedure within the restricted master heuristic. Third, we employed the ALNS to solve the elementary shortest path problem with resource and time synchronization constraints, which are encountered as a subproblem to populate the route sets heuristically within our restricted master heuristic.

On a structured set of benchmark instances, we show the performance of our suite of solution approaches. We show that integration of the ALNS in the restricted master heuristic reduces computation time by up to 98% while maintaining root-node optimality. Furthermore, a version of our restricted master heuristic that only uses the ALNS to populate the route sets finds a root-node "lower bounds" that are within 0.04% of the true lower bounds. Additionally, we demonstrate that the ALNS algorithm yields competitive solutions for medium- to large-scale benchmark problems. Lastly, we applied the ALNS algorithm in a case study at a major European logistics service provider. On average, over 24 days of operations, we show that costs can be saved by 17.4%. We utilize our algorithms to analyze a common practical approach employed by logistics service providers operating from multiple depots. Specifically, they assign a specific geographical area to each depot, thereby treating the multi-depot problem as multiple single-depot alternatives. We demonstrate that this practice results in a 4.9% cost increase at our industry partner.

There are several avenues for further research. First and foremost, we hope that our publicly available real-life instances will spark the interest of researchers in studying large-scale *rich* pickup-and-delivery problems. More real-life elements need to be considered for the real-time deployment of algorithms in such large-scale systems, including time-dependent travel times and stochastic travel and service times. A promising research direction is to effectively address uncertainty within large-scale routing problems. Another opportunity lies in integrating the intra-logistics operations at the cross docks (i.e., the depots in this study) with planning the pickup-and-delivery routes, as otherwise the loading and unloading of trucks at warehouses will be subject to congestion and complicate efficient consolidation.

CRediT authorship contribution statement

Casper H. Bazelmans: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Siem L. ter Braake:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation. **Albert H. Schrotenboer:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Funding acquisition, Formal analysis, Conceptualization. **Rolf N. van Lieshout:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Formal analysis, Conceptualization. **Tom Van Woensel:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Conceptualization.

Funding

A. H. Schrotenboer received support from the Dutch Science Foundation (Nederlandse Organisatie voor Wetenschappelijk Onderzoek; NWO) through [Grant VI.Veni.211E.043].

Acknowledgments

We thank SURF www.surf.nl for the support in using the National Supercomputer Snellius.

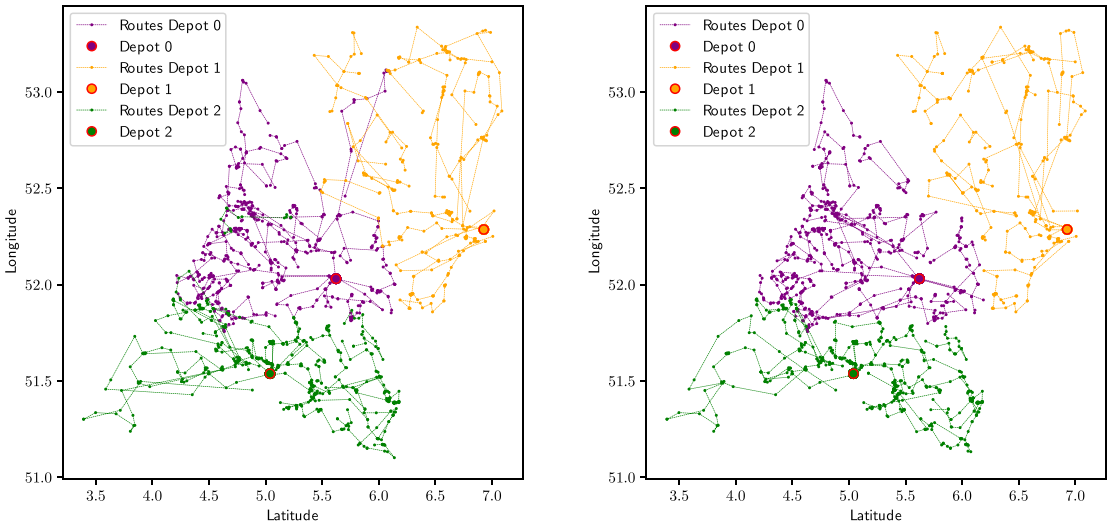


Fig. 4. Impact of depot service area restrictions.
Note. Solutions for instance NL_1 without service area restrictions (left) and a scenario with service area restrictions (right). For clarity, the edges connecting to the depots are not visualized. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Appendix A. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.ejor.2025.07.070.

References

Angelelli, E., & Mansini, R. (2002). The Vehicle Routing Problem with Time Windows and Simultaneous Pick-up and Delivery. In *Quantitative approaches to distribution logistics and supply chain management*: *519*, (pp. 249–267). Berlin: Springer, http://dx.doi.org/10.1007/978-3-642-56183-2_15.

Arnold, F., Gendreau, M., & Sörensen, K. (2019). Efficiently solving very large-scale routing problems. *Computers and Operations Research*, *107*, 32–42. http://dx.doi.org/10.1016/j.cor.2019.03.006.

Avci, M., & Topaloglu, S. (2016). A hybrid metaheuristic algorithm for heterogeneous vehicle routing problem with simultaneous pickup and delivery. *Expert Systems with Applications*, *53*, 160–171. http://dx.doi.org/10.1016/j.eswa.2016.01.038.

Berbeglia, G., Cordeau, J.-F., Gribkovskaia, I., & Laporte, G. (2007). Static pickup and delivery problems: a classification scheme and survey. *Top*, *15*, 1–31. http://dx.doi.org/10.1007/s11750-007-0009-0.

Bustos-Coral, D., & Costa, A. M. (2025). Adaptive large neighborhood search for drayage routing problems involving longer combination vehicles. *Computers & Operations Research*, *173*, Article 106826. http://dx.doi.org/10.1016/j.cor.2024.106826.

Cavaliere, F., Accorsi, L., Laganà, D., Musmanno, R., & Vigo, D. (2024). An efficient heuristic for very large-scale vehicle routing problems with simultaneous pickup and delivery. *Transportation Research Part E: Logistics and Transportation Review*, *186*, http://dx.doi.org/10.1016/j.tre.2024.103550.

Cortés, C. E., Matamala, M., & Contardo, C. (2010). The pickup and delivery problem with transfers: Formulation and a branch-and-cut solution method. *European Journal of Operational Research*, *200*(3), 711–724. http://dx.doi.org/10.1016/j.ejor.2009.01.022.

Dell'Amico, M., Righini, G., & Salani, M. (2006). A branch-and-price approach to the vehicle routing problem with simultaneous distribution and collection. *Transportation Science*, *40*(2), 235–247. http://dx.doi.org/10.1287/trsc.1050.0118.

Drexl, M. (2012). Synchronization in vehicle routing - A survey of VRPs with multiple synchronization constraints. *Transportation Science*, *46*(3), 297–316. http://dx.doi.org/10.1287/trsc.1110.0400.

Ghilas, V., Cordeau, J.-F., Demir, E., & Van Woensel, T. (2018). Branch-and-price for the pickup and delivery problem with time windows and scheduled lines. *Transportation Science*, *52*(5), 1191–1210. http://dx.doi.org/10.1287/trsc.2017.0798.

Ghilas, V., Demir, E., & Van Woensel, T. (2016a). An adaptive large neighborhood search heuristic for the Pickup and Delivery Problem with Time Windows and Scheduled Lines. *Computers and Operations Research*, *72*, 12–30. http://dx.doi.org/10.1016/j.cor.2016.01.018.

Ghilas, V., Demir, E., & Van Woensel, T. (2016b). The pickup and delivery problem with time windows and scheduled lines. *INFOR: Information Systems and Operational Research*, *54*(2), 147–167. http://dx.doi.org/10.1080/03155986.2016.1166793.

Goksal, F. P., Karaoglan, I., & Altiparmak, F. (2013). A hybrid discrete particle swarm optimization for vehicle routing problem with simultaneous pickup and delivery. *Computers & Industrial Engineering*, *65*(1), 39–53. http://dx.doi.org/10.1016/j.cie.2012.01.005.

Homberger, J., & Gehring, H. (2005). A two-phase hybrid metaheuristic for the vehicle routing problem with time windows. *European Journal of Operational Research*, *162*(1), 220–238. http://dx.doi.org/10.1016/j.ejor.2004.01.027.

Irnich, S., & Desaulniers, G. (2005). Shortest path problems with resource constraints. In *Column generation* (pp. 33–65). Springer US, http://dx.doi.org/10.1007/0-387-25486-2_2.

Koç, c., Laporte, G., & Tükenmez, İ. (2020). A review of vehicle routing with simultaneous pickup and delivery. *Computers and Operations Research*, *122*, http://dx.doi.org/10.1016/j.cor.2020.104987.

Lyu, Z., & Yu, A. J. (2023). The pickup and delivery problem with transshipments: Critical review of two existing models and a new formulation. *European Journal of Operational Research*, *305*(1), 260–270. http://dx.doi.org/10.1016/j.ejor.2022.05.053.

Min, H. (1989). The multiple vehicle routing problem with simultaneous delivery and pick-up points. *Transportation Research Part A: General*, *23*(5), 377–386. http://dx.doi.org/10.1016/0191-2607(89)90085-X.

Nagy, G., & Salhi, S. d. (2005). Heuristic algorithms for single and multiple depot vehicle routing problems with pickups and deliveries. *European Journal of Operational Research*, *162*(1), 126–141. http://dx.doi.org/10.1016/j.ejor.2002.11.003.

Ropke, S., & Pisinger, D. (2006). An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. *Transportation Science*, *40*(4), 455–472. http://dx.doi.org/10.1287/trsc.1050.0135.

Santini, A., Ropke, S., & Hvattum, L. M. (2018). A comparison of acceptance criteria for the adaptive large neighbourhood search metaheuristic. *Journal of Heuristics*, *24*, 783–815.

Sonntag, D. R., Schrotenboer, A. H., & Kiesmüller, G. P. (2023). Stochastic inventory routing with time-based shipment consolidation. *European Journal of Operational Research*, *306*(3), 1186–1201. http://dx.doi.org/10.1016/j.ejor.2022.07.049.

Subramanian, A., Uchoa, E., & Ochi, L. S. (2013). A hybrid algorithm for a class of vehicle routing problems. *Computers and Operations Research*, *40*(10), 2519–2531. http://dx.doi.org/10.1016/j.cor.2013.01.013.

Subramanian, A., Uchoa, E., Pessoa, A. A., & Ochi, L. S. (2013). Branch-cut-and-price for the vehicle routing problem with simultaneous pickup and delivery. *Optimization Letters*, *7*(7), 1569–1581. http://dx.doi.org/10.1007/s11590-012-0570-9.

Uchoa, E., Pecin, D., Pessoa, A., Poggi, M., Vidal, T., & Subramanian, A. (2017). New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, *257*(3), 845–858. http://dx.doi.org/10.1016/j.ejor.2016.08.012.